Competitive Programming

Primes and Factoring

Primes and Factoring

- Common task or sub-task for problems
- Chance to work with numbers larger than 32 bits

Primality Testing … the dumb way

```
bool isPrime( long val ) {
   for ( long f = 2; f < val; f++ )
      if ( val % f == 0 )
         return false;
   return true;
}
```

Try all smaller values as factors.

See if this factor divides val evenly.

Primality Testing … the dumb way

```
bool isPrime( long val ) {
   for ( long f = 2; f < val; f++ )
      if ( val % f == 0 )
         return false;
   return true;
}
```

This will be too slow, if val is 10^9 or greater.

This divide operation can be a little expensive.

- Multiply
  - 10^9 int multiply operations : about the same cost as addition
  - 10^9 long multiplies : about the same cost as addition
- Divide
  - 10^9 int divide or mod operations : 1.7 seconds
  - 10^9 long divide or mod operations : 7 seconds

Primality Testing … a little smarter

```
bool isPrime( long val ) {
   for ( long f = 2; f * f <= val; f++ )
      if ( val % f == 0 )
         return false;
   return true;
}
```

Half the factors will be smaller than $\sqrt{val}$

Almost all primes are odd

```
bool isPrime( long val ) {
   if ( val % 2 == 0 )
       return val == 2;
   for ( long f = 3; f * f <= val; f += 2 )
      if ( val % f == 0 )
         return false;
   return true;
}
```

Handle even numbers as a special case.

Just consider odd factors for everything else.s

If you have a factor … you have a prime factor.

```
bool isPrime( long val ) {
   for ( long i = 0; plist[ i ] * plist[ i ] <= val; i += 2 )
      if ( val % plist[ i ] == 0 )
         return false;
   return true;
}
```

Need to make sure you have enough primes to get out of this loop.

Could be a lot faster; there are about x / ln(x) primes smaller than x.

Finding Lots of Primes

- Sieve of Eratosthenes

```
vector< char > pflag( 1000004, true );
vector< long > plist;

pflag[ 0 ] = pflag[ 1 ] = false;
for ( int i = 0; i < pflag.size(); i++ )
   if ( plist[ i ] ) {
     for ( int j = i * 2; j < pflag.size(); j += i )
        pflag[ j ] = false;
     plist.add( i );
   }
```

Set and list of primes

If i is a prime …

… then none of its multiples are.

Put it on our prime list.

Finding Lots of Primes

- Sieve of Eratosthenes

C++ has a bitset, which could make this more efficient.

```
vector< char > pflag( 1000004, true );
vector< long > plist;

pflag[ 0 ] = pflag[ 1 ] = false;
for ( int i = 0; i < pflag.size(); i
   if ( plist[ i ] ) {
     for ( int j = i * i; j < pflag.size(); j += i )
        pflag[ j ] = false;
     plist.add( i );
   }
```

A little bit faster. We've already killed all smaller multiples of i.

Primality Testing **Large** values

- This approach would let us test a moderate number of values, no larger than 10^{12}
- For larger values, there's a probabilistic primality tester
  - It uses Fermat's little theorem (on each test, it catches most non-prime number with a 50-percent chance)
  - This is used in RSA to find large primes
  - And, it's built into the java BigInteger class, so you don't have to implement it yourself.
- There's a Kattis problem that this will solve easily: Primality

Factoring … the dumb way

```
void factor( long val, vector< long > factors ) {
   for ( long f = 1; f <= val; f++ )
      if ( val % f == 0 )
         factors.add( f );
}
```

Bad idea, counting all the way up to val.

(most) Factors come in pairs

```
void factor( long val, vector< long > factors ) {
  for ( long f = 1; f * f <= val; f++ )
    if ( val % f == 0 ) {
      factors.add( f );
      factors.add( val / f );
    }
}
```

Get the small factor.

Get the larger factor.

Oops, two copies of the same factor if val is a perfect square
(easy to fix)

Perfect Squares

```
void factor( long val, vector< long > factors ) {
   long f = 1;
   while( f * f < val )
      if ( val % f == 0 ) {
         factors.add( f );
         factors.add( val / f );
      }
   if ( f * f == val )
      factors.add( val );
}
```

OK, handle perfect squares as a special case.

Prime Factorization

```
void primeFactor( long val, vector< long > factors ) {
  long f = 2;
  while ( f * f <= val ) {
    while ( val % f == 0 ) {
      factors.add( f );
      val /= f;
    }

    f += 1;
  }

  if ( val != 1 )
    factors.add( val );
}
```

Save the factor and divide it out.

Whatever's left in val must be prime.